

13 – Mutual Exclusion, Synchronization, Deadlocks

How to ensure resource exclusivity and how to synchronize tasks?

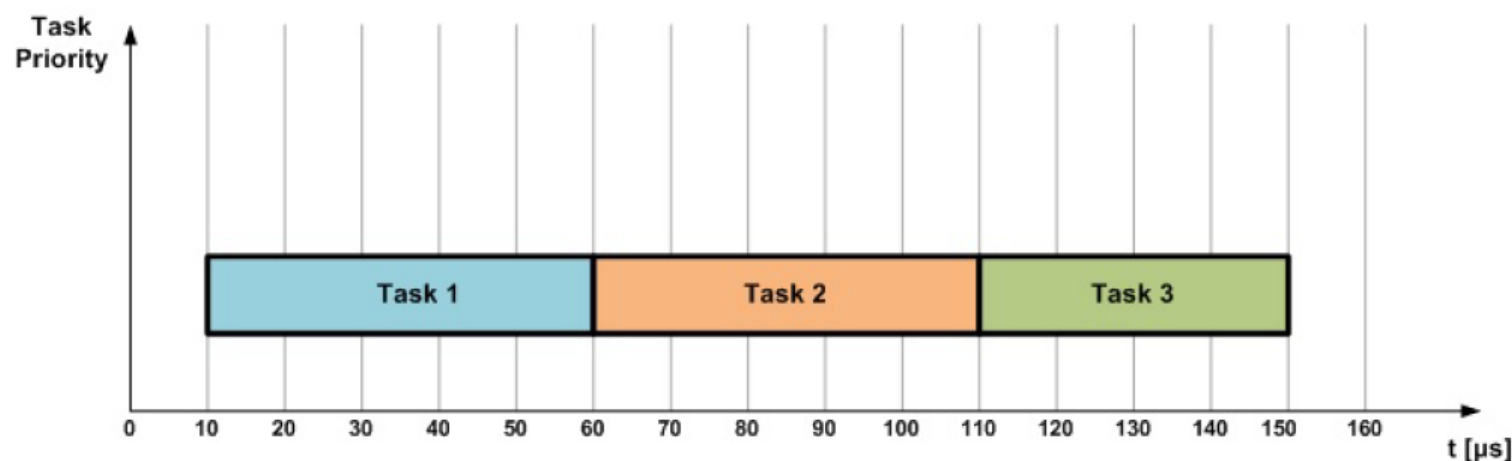
Jérôme Landré – jerome.landre@ju.se

OUTLINE

- 1) Preemptive Scheduling
- 2) Race Condition
- 3) Mutual Exclusion
- 4) Mutex vs. Semaphore
- 5) Synchronization of Threads
- 6) Deadlock
- 7) Priority Inversion

1) PREEMPTIVE SCHEDULING

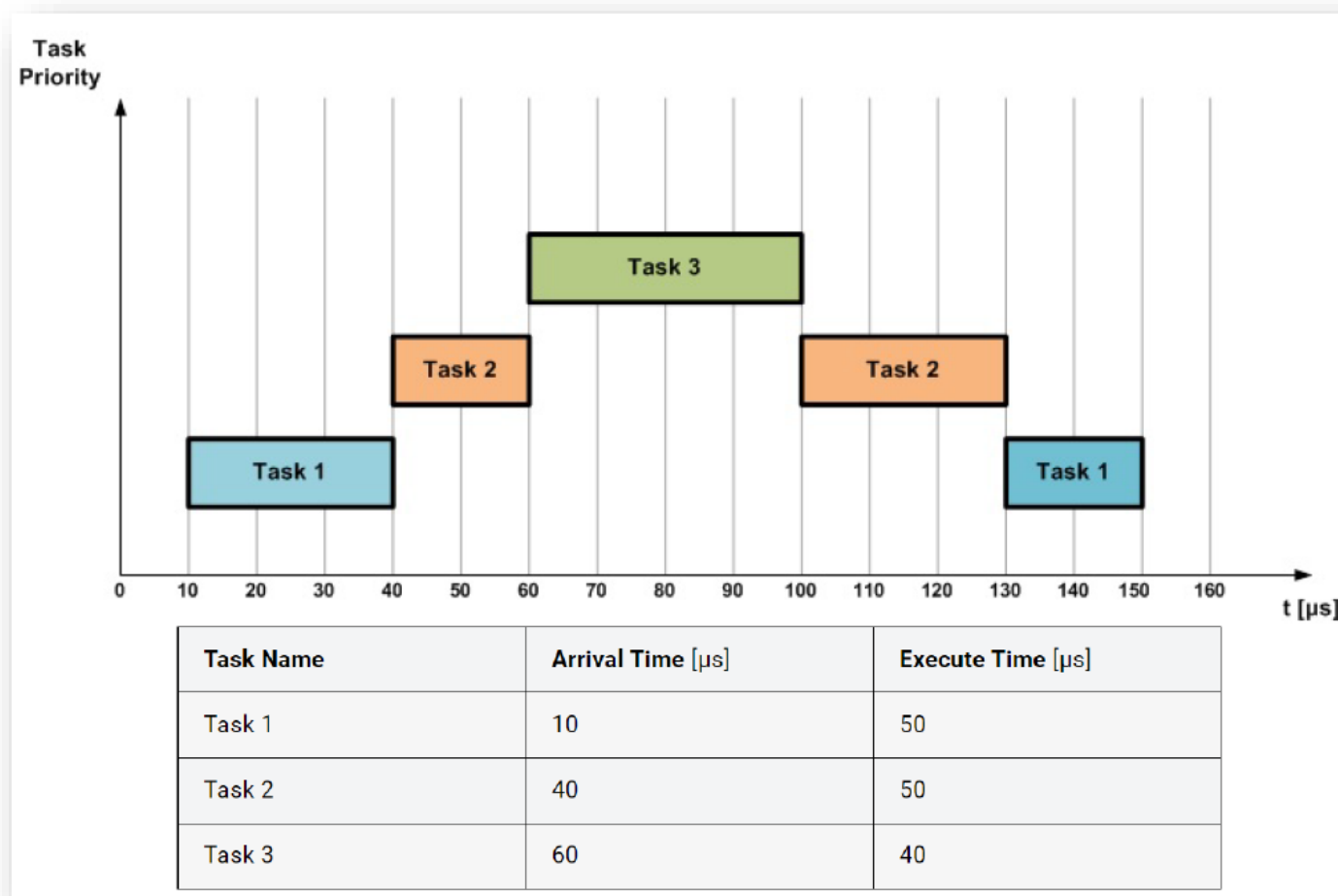
- In case of **non-preemptive** scheduling, a task cannot be interrupted:



Task Name	Arrival Time [μs]	Execute Time [μs]
Task 1	10	50
Task 2	40	50
Task 3	60	40

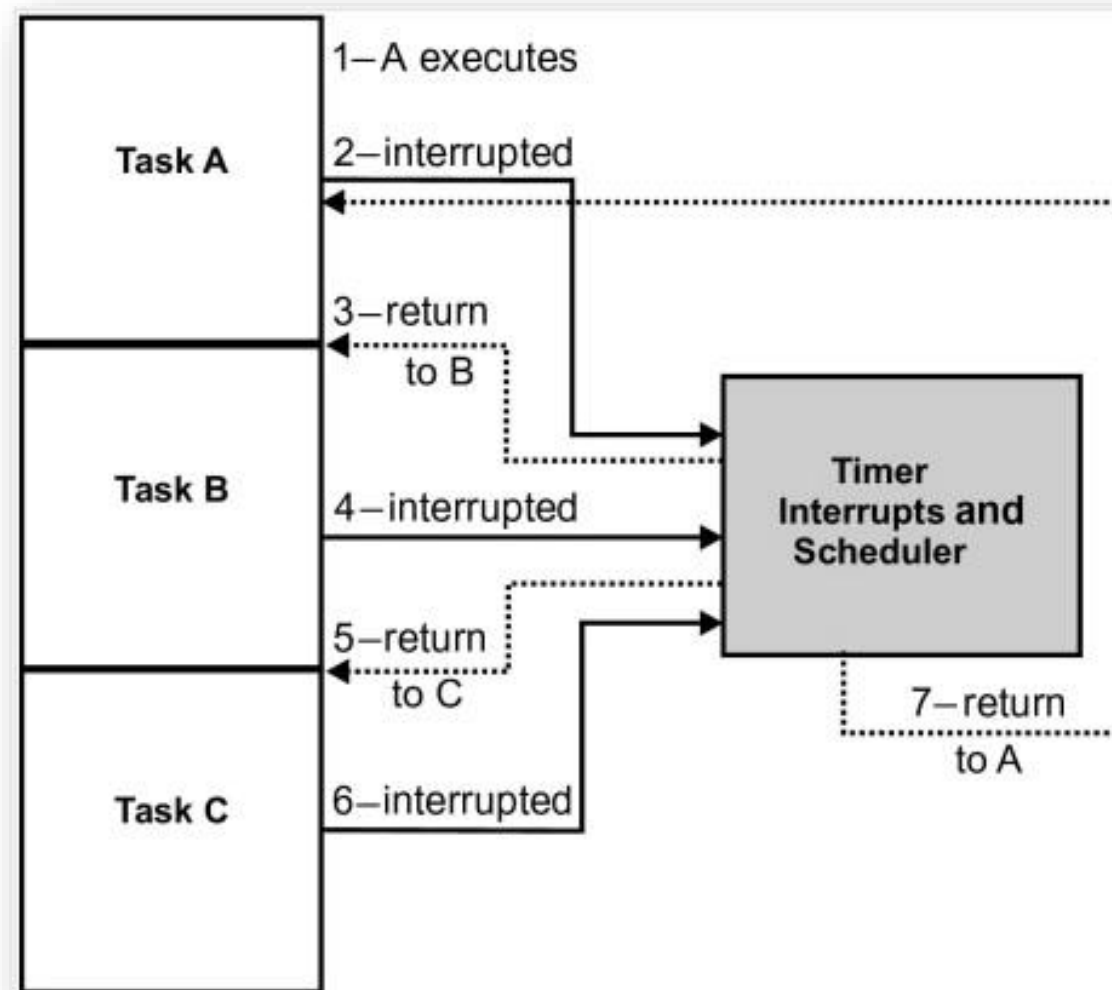
1) PREEMPTIVE SCHEDULING

- In case of **preemptive** scheduling, a task can be interrupted:



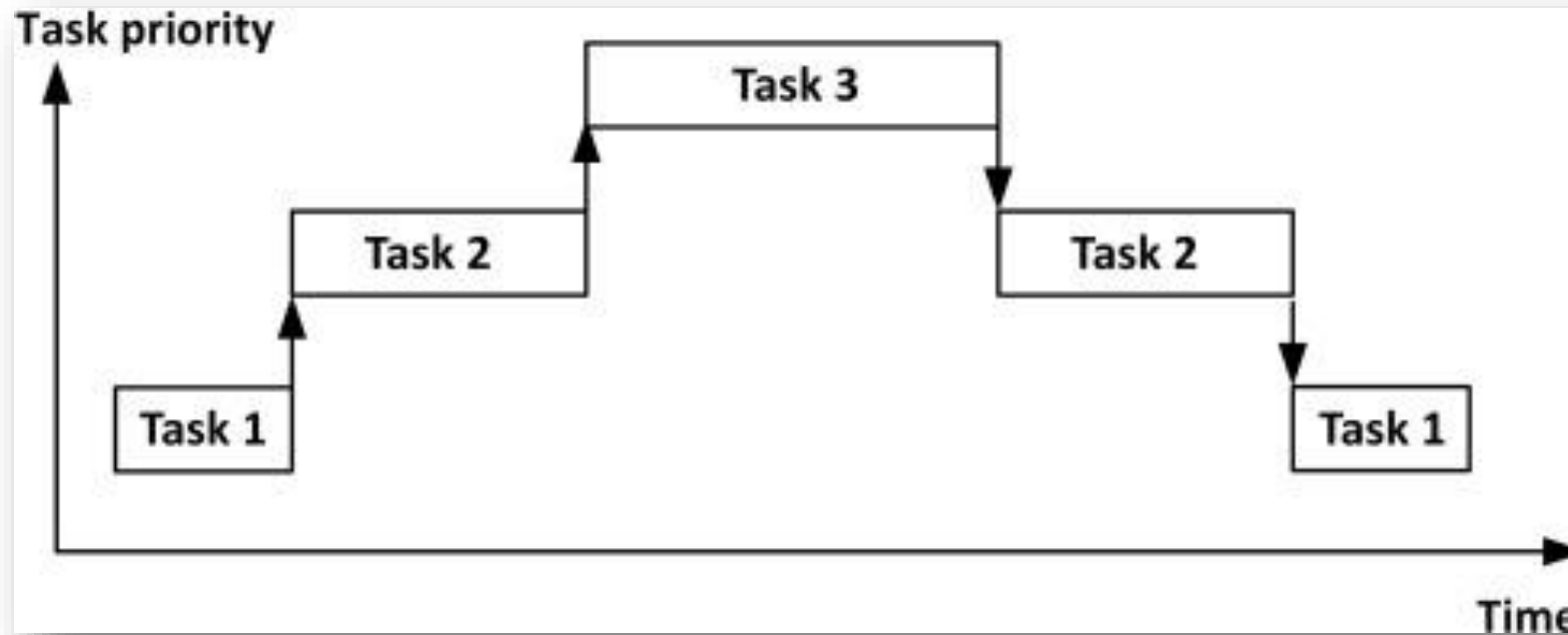
1) PREEMPTIVE SCHEDULING

- In case of **preemptive** scheduling, a task can be interrupted:



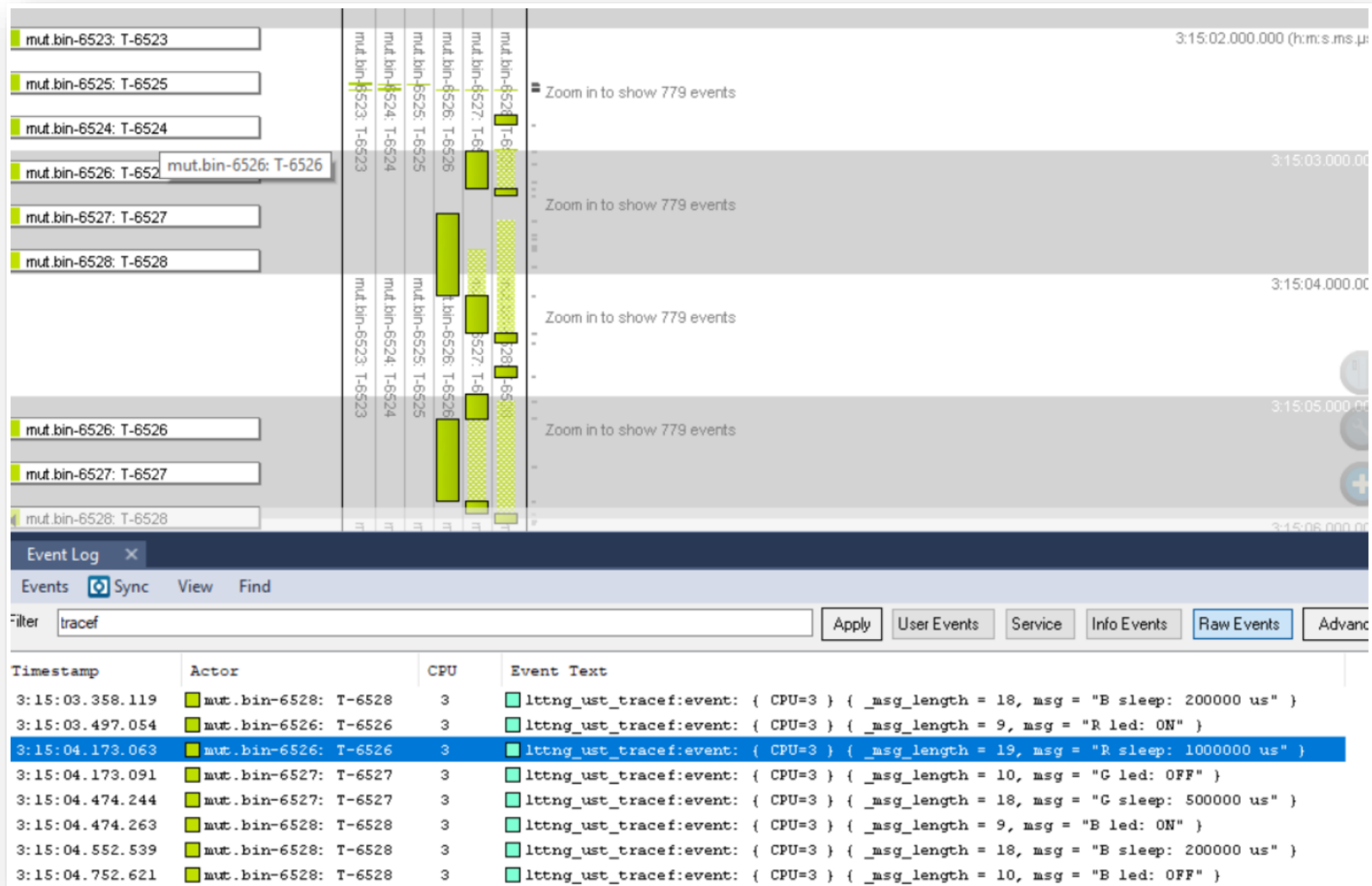
1) PREEMPTIVE SCHEDULING

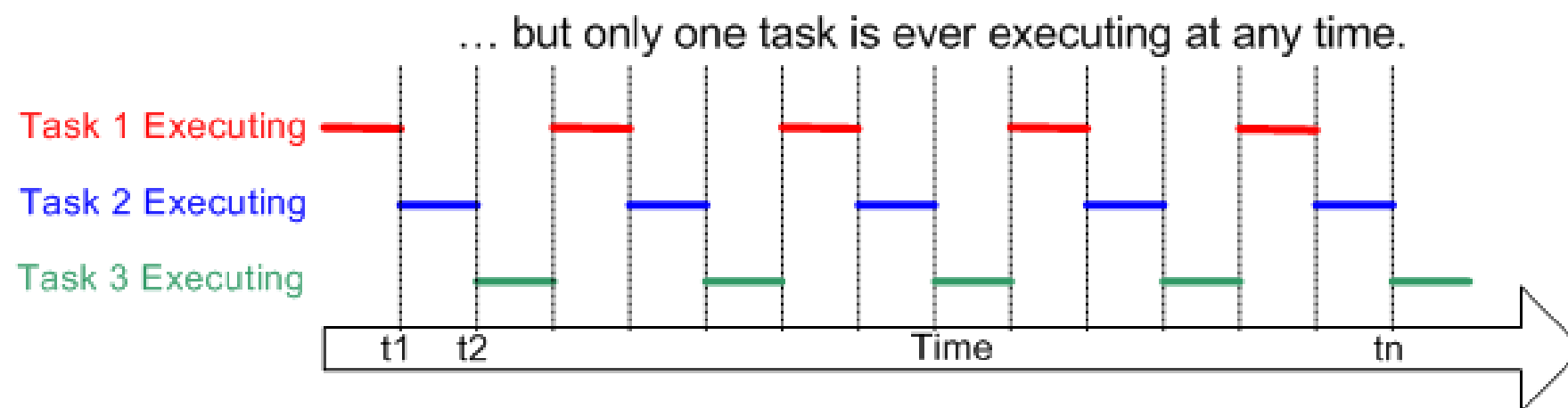
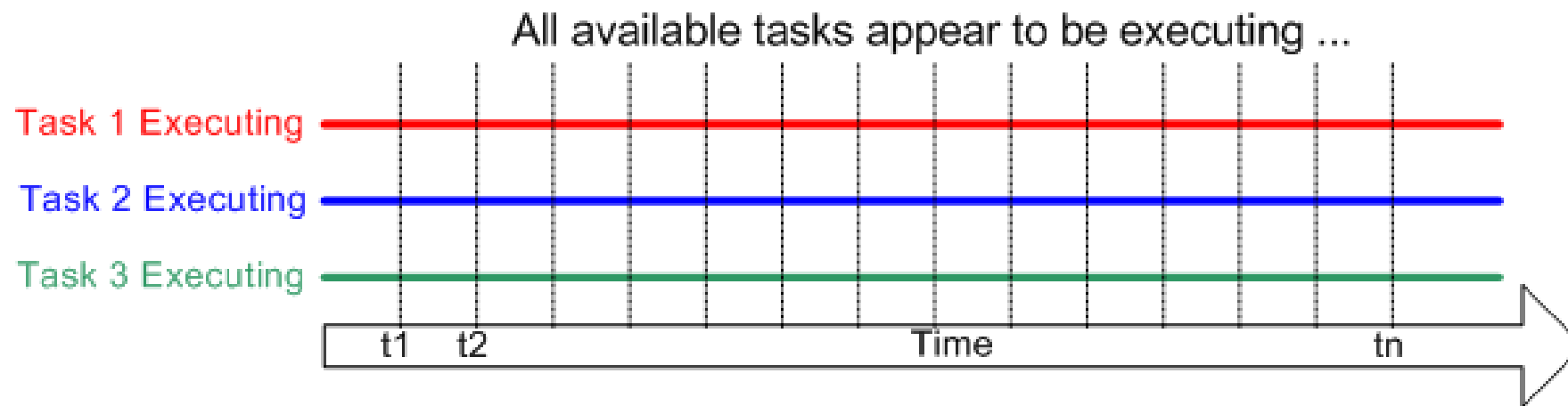
- In case of **preemptive** scheduling, a task can be interrupted:

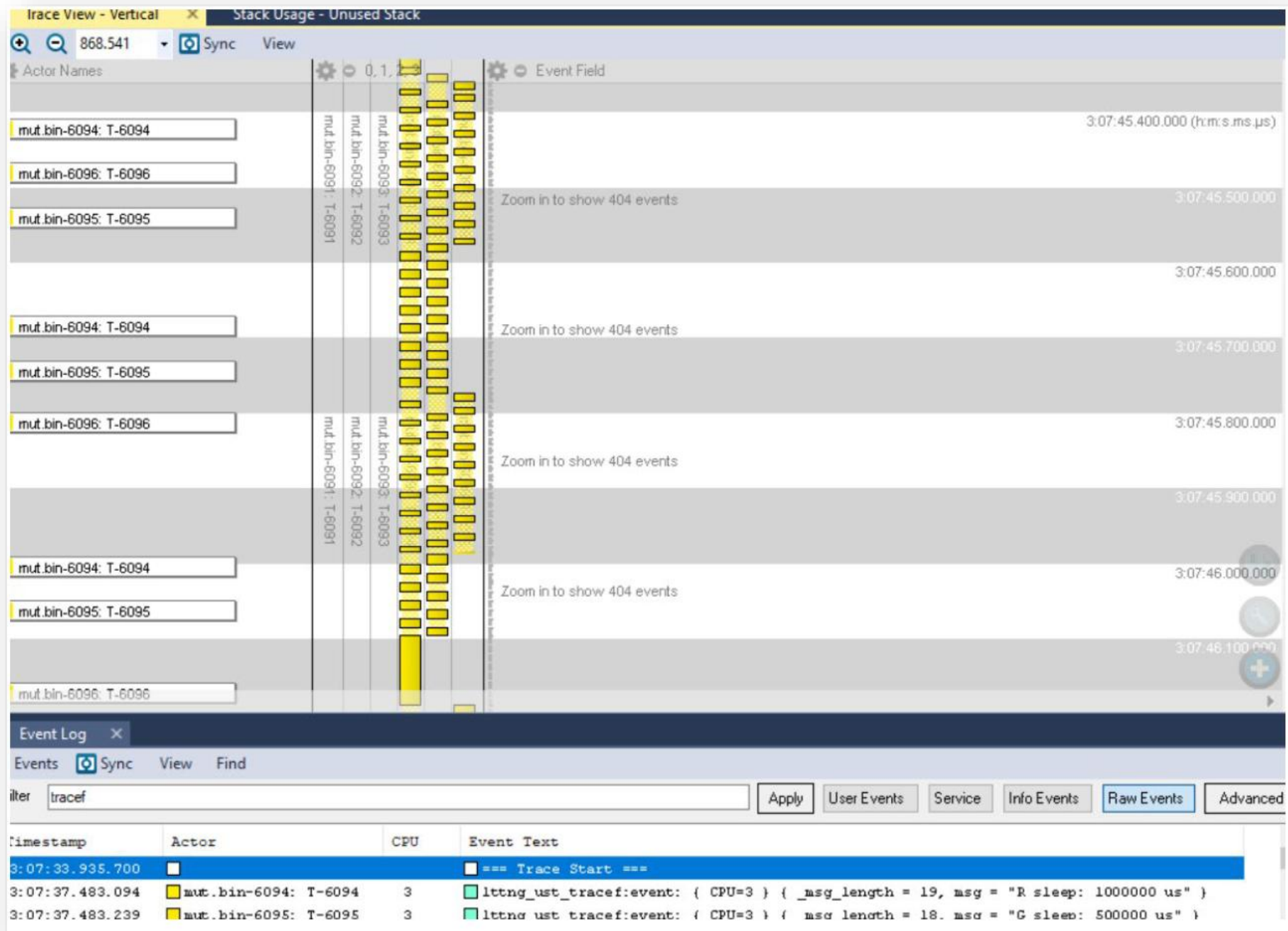


1) PREEMPTIVE SCHEDULING

”Interruptions drive the operating system”







1) PREEMPTIVE SCHEDULING

- Linux scheduling:
 - Non-realtime
 - SCHED_OTHER (round robin)
 - Realtime
 - SCHED_FIFO (priority based)
 - SCHED_RR (priority based with round robin)

2) RACE CONDITION

- An example to begin with...

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_THREADS 8
5 #define ITERATIONS 1000000
6
7 int shared_counter = 0;
8
9 void* increment_counter(void* arg) {
10     for (int i = 0; i < ITERATIONS; i++) {
11         shared_counter++;
12     }
13     return NULL;
14 }
15
16 int main() {
17     // Define an array of threads
18     pthread_t threads[NUM_THREADS];
19
20     // Create NUM_THREADS threads
21     for (int i = 0; i < NUM_THREADS; i++) {
22         pthread_create(&threads[i], NULL, increment_counter, NULL);
23     }
24
25     // Wait for all threads to finish
26     for (int i = 0; i < NUM_THREADS; i++) {
27         pthread_join(threads[i], NULL);
28     }
29
30     printf("Final counter value: %d (expected: %d)\n",
31           shared_counter, NUM_THREADS * ITERATIONS);
32
33     return 0;
34 }
35
```

2) RACE CONDITION

- An example to begin with...

```
jerome@jerome-virtualbox:~/c-code/race-condition$ gcc -o rc rc.c
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4180363 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4100869 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4508214 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 3913961 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4164362 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4262658 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4267154 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$
```

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_THREADS 8
5 #define ITERATIONS 1000000
6
7 int shared_counter = 0;
8
9 void* increment_counter(void* arg) {
10     for (int i = 0; i < ITERATIONS; i++) {
11         shared_counter++;
12     }
13     return NULL;
14 }
15
16 int main() {
17     // Define an array of threads
18     pthread_t threads[NUM_THREADS];
19
20     // Create NUM_THREADS threads
21     for (int i = 0; i < NUM_THREADS; i++) {
22         pthread_create(&threads[i], NULL, increment_counter, NULL);
23     }
24
25     // Wait for all threads to finish
26     for (int i = 0; i < NUM_THREADS; i++) {
27         pthread_join(threads[i], NULL);
28     }
29
30     printf("Final counter value: %d (expected: %d)\n",
31           shared_counter, NUM_THREADS * ITERATIONS);
32
33     return 0;
34 }
35
```

2) RACE CONDITION

- An example to begin with...

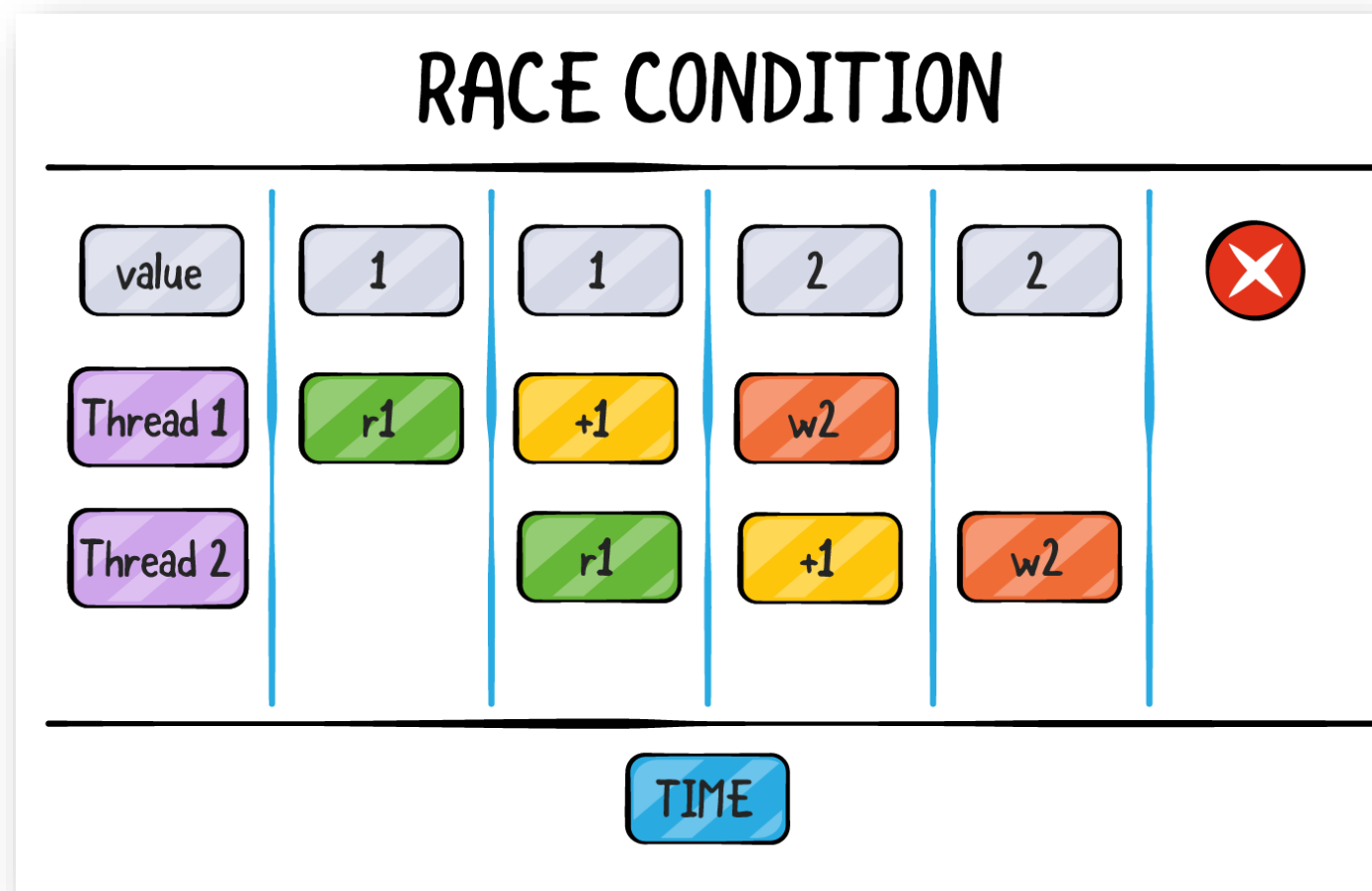
```
jerome@jerome-virtualbox:~/c-code/race-condition$ gcc -o rc rc.c
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4180363 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4100869 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4508214 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 3913961 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4164362 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4262658 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc
Final counter value: 4267154 (expected: 8000000)
jerome@jerome-virtualbox:~/c-code/race-condition$
```

Why?

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_THREADS 8
5 #define ITERATIONS 1000000
6
7 int shared_counter = 0;
8
9 void* increment_counter(void* arg) {
10     for (int i = 0; i < ITERATIONS; i++) {
11         shared_counter++;
12     }
13     return NULL;
14 }
15
16 int main() {
17     // Create an array of threads
18     pthread_t threads[NUM_THREADS];
19
20     // Create NUM_THREADS threads
21     for (int i = 0; i < NUM_THREADS; i++) {
22         pthread_create(&threads[i], NULL, increment_counter, NULL);
23     }
24
25     // Wait for all threads to finish
26     for (int i = 0; i < NUM_THREADS; i++) {
27         pthread_join(threads[i], NULL);
28     }
29
30     printf("Final counter value: %d (expected: %d)\n",
31           shared_counter, NUM_THREADS * ITERATIONS);
32
33     return 0;
34 }
35
```

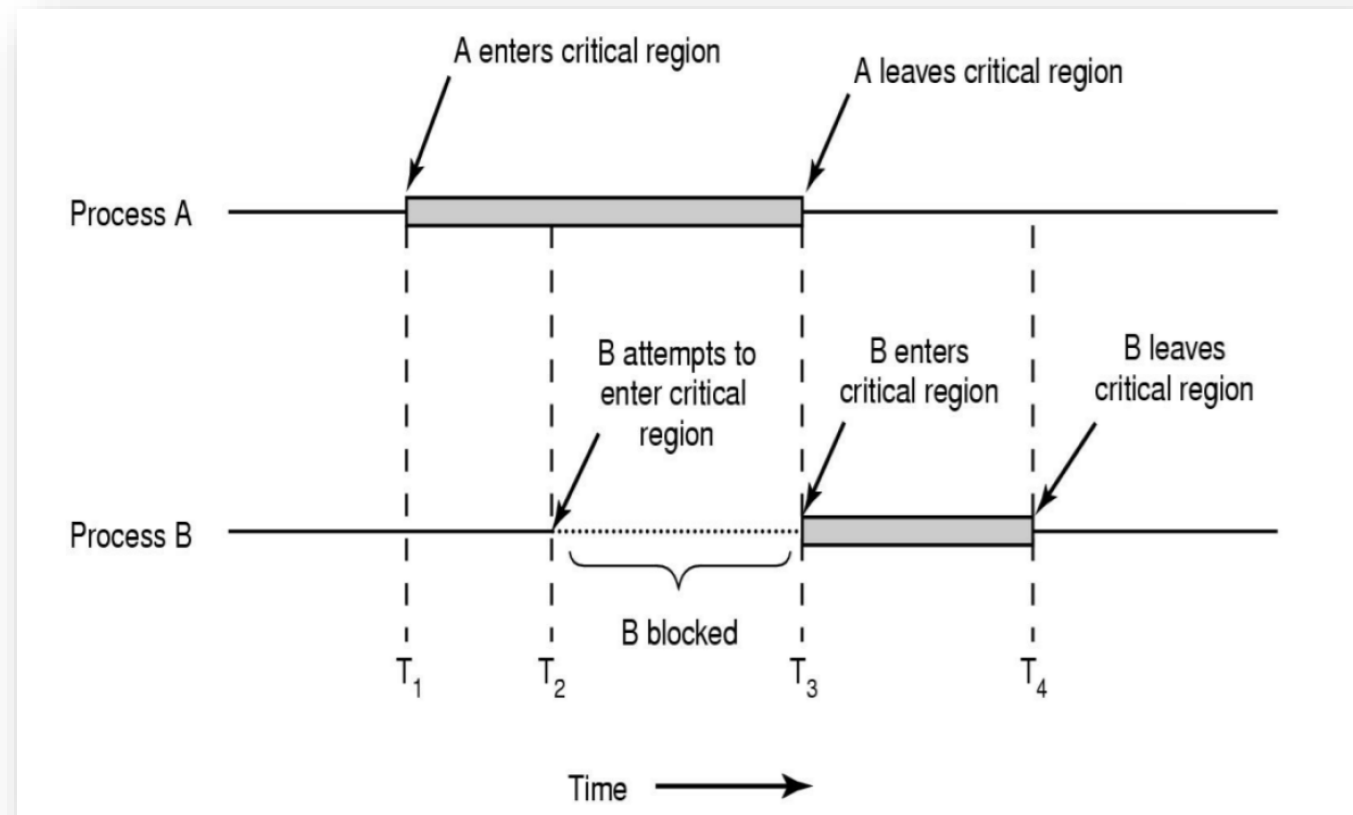
2) RACE CONDITION

- Race condition:
 - With the execution in parallel of multiple tasks accessing the same resources, we must be careful on the values of the resources!



3) MUTUAL EXCLUSION

- Critical Region:
 - When using a resource, we must define critical region to avoid tasks collisions on the same resource



3) MUTUAL EXCLUSION

- The process/thread can also wait until a device is available
- It is the simplest way to know if a device is available
- Spin locks / "busy waiting" / polling

```
while (lock is unavailable)
    continue; // try again
return success;
```

3) MUTUAL EXCLUSION

- **Use Cases**

- **Short Critical Sections:** Spin locks are efficient when the critical section is very short, and the thread is unlikely to wait long.
- **Multiprocessor Systems:** Spin locks are useful in multiprocessor environments where the lock holder might be running on another CPU.
- **Kernel-Level Synchronization:** Often used in operating system kernels to protect data structures like linked lists, hash tables, or queues.

3) MUTUAL EXCLUSION

- **Advantages**

- **No Context Switching**: Since the thread does not sleep, there is no overhead of context switching or scheduling.
- **Low Latency**: Spin locks are fast to acquire and release, making them suitable for performance-critical code.
- **Useful in Non-Preemptive Kernels**: In kernels where threads are not preempted, spin locks prevent deadlocks that could occur with sleeping locks.

3) MUTUAL EXCLUSION

- **Disadvantages**
 - **CPU Waste**: If the lock is held for a long time, spinning threads waste CPU cycles, leading to inefficient resource usage.
 - **Priority Inversion**: Spin locks can exacerbate priority inversion, where a low-priority thread holds a lock needed by a high-priority thread.
 - **Not Suitable for User Space**: In user-space applications, spin locks are generally avoided because they can lead to poor system responsiveness.

- Spin locks:
Implementation

```
; Intel syntax

locked:                                ; The lock variable. 1 = locked, 0 = unlocked.
    dd    0

spin_lock:
    mov    eax, 1                    ; Set the EAX register to 1.
    xchg   eax, [locked]             ; Atomically swap the EAX register with
                                    ; the lock variable.
                                    ; This will always store 1 to the lock, leaving
                                    ; the previous value in the EAX register.
    test   eax, eax                 ; Test EAX with itself. Among other things, this will
                                    ; set the processor's Zero Flag if EAX is 0.
                                    ; If EAX is 0, then the lock was unlocked and
                                    ; we just locked it.
                                    ; Otherwise, EAX is 1 and we didn't acquire the lock.
    jnz    spin_lock                ; Jump back to the MOV instruction if the Zero Flag is
                                    ; not set; the lock was previously locked, and so
                                    ; we need to spin until it becomes unlocked.
    ret                               ; The lock has been acquired, return to the calling
                                    ; function.

spin_unlock:
    xor    eax, eax                 ; Set the EAX register to 0.
    xchg   eax, [locked]             ; Atomically swap the EAX register with
                                    ; the lock variable.
    ret                               ; The lock has been released.
```

3) MUTUAL EXCLUSION

- Spin locks / "busy waiting" / polling

`pthread_spin_lock()` Syntax

```
int  pthread_spin_lock(pthread_spinlock_t *lock);
```

```
#include <pthread.h>
```

```
pthread_spinlock_t lock;
```

```
int ret;
```

```
ret = pthread_spin_lock(&lock); /* lock the spinlock */
```

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4
5 #define NUM_THREADS 8
6 #define ITERATIONS 100000000
7
8 int counter = 0;
9 pthread_spinlock_t spinlock;
10
11 void* increment(void* arg) {
12     for (int i = 0; i < ITERATIONS; i++) {
13         pthread_spin_lock(&spinlock);
14         counter++;
15         pthread_spin_unlock(&spinlock);
16     }
17     return NULL;
18 }
19
```

```
19
20 int main() {
21     pthread_t threads[NUM_THREADS];
22
23     // Initialize the spin lock
24     if (pthread_spin_init(&spinlock, PTHREAD_PROCESS_PRIVATE) != 0) {
25         perror("Failed to initialize spin lock");
26         exit(EXIT_FAILURE);
27     }
28
29     // Create threads
30     for (int i = 0; i < NUM_THREADS; i++) {
31         if (pthread_create(&threads[i], NULL, increment, NULL) != 0) {
32             perror("Failed to create thread");
33             exit(EXIT_FAILURE);
34         }
35     }
36
37     // Wait for all threads to complete
38     for (int i = 0; i < NUM_THREADS; i++) {
39         if (pthread_join(threads[i], NULL) != 0) {
40             perror("Failed to join thread");
41             exit(EXIT_FAILURE);
42         }
43     }
44
45     // Destroy the spin lock
46     pthread_spin_destroy(&spinlock);
47
48     printf("Expected: %d, Actual: %d\n", NUM_THREADS * ITERATIONS, counter);
49     return 0;
50 }
```



```
jerome@jerome-virtualbox:~/c-code/race-condition$ gcc rc-spin-lock-pthread.c -o rc-spin-lock-pthread -lpthread
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-spin-lock-pthread
Expected: 800000000, Actual: 800000000
jerome@jerome-virtualbox:~/c-code/race-condition$
```

- Problem solved!

100.0% Tasks: 98, 363 thr, 86 kthr; 2 running
100.0% Load average: 0.88 0.52 0.40
1.22G/3.82G Uptime: 00:14:11
0K/512M

Main	I/O	PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
		12890	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:03.27	./rc-spin-lock-pthr
		12891	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:02.75	./rc-spin-lock-pthr
		12892	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:01.77	./rc-spin-lock-pthr
		12893	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:03.26	./rc-spin-lock-pthr
		12894	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:01.82	./rc-spin-lock-pthr
		12895	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:02.49	./rc-spin-lock-pthr
		12896	jerome	20	0	68248	1508	1380	R	200.0	0.0	0:02.20	./rc-spin-lock-pthr
		12889	jerome	20	0	68248	1508	1380	R	171.4	0.0	0:01.61	./rc-spin-lock-pthr
		12767	jerome	20	0	14260	5212	3804	R	28.6	0.1	0:07.53	htop
		10240	jerome	20	0	3742M	306M	104M	S	19.0	7.8	0:00.33	/snap/firefox/7423/
		1	root	20	0	23040	13904	9424	S	0.0	0.3	0:02.94	/sbin/init splash
		261	root	19	-1	66968	17592	16440	S	0.0	0.4	0:00.33	/usr/lib/systemd/sy
		336	root	20	0	30360	8272	4944	S	0.0	0.2	0:00.18	/usr/lib/systemd/sy
		372	systemd-re	20	0	21812	12880	10448	S	0.0	0.3	0:00.29	/usr/lib/systemd/sy
		376	systemd-ti	20	0	91048	7820	6924	S	0.0	0.2	0:00.04	/usr/lib/systemd/sy
		525	systemd-ti	20	0	91048	7820	6924	S	0.0	0.2	0:00.00	/usr/lib/systemd/sy
		669	root	20	0	308M	7788	7148	S	0.0	0.2	0:00.06	/usr/libexec/accoun
		672	avahi	20	0	8612	4424	4040	S	0.0	0.1	0:00.09	avahi-daemon: runni
		674	root	20	0	12304	3100	2844	S	0.0	0.1	0:00.00	/usr/sbin/cron -f -
		675	messagebus	20	0	11180	6536	4488	S	0.0	0.2	0:00.78	@dbus-daemon --syst
		677	root	20	0	308M	7788	7148	S	0.0	0.2	0:00.00	/usr/libexec/accoun

F1 Help F2 Setup F3 Search F4 Filter F5 Tree F6 SortBy F7 Nice F8 Nice + F9 Kill F10 Quit

3) MUTUAL EXCLUSION

- **Mutex**

- Tries to take a lock (mutex). If already taken, move thread to blocked list and run thread from ready list, otherwise lock the mutex. Do critical section. Return lock.
- Solutions often depends on “Atomic instruction” (Test and Set)

```
#include <pthread.h>
```

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

3) MUTUAL EXCLUSION

- Problem solved!

```
jerome@jerome-virtualbox:~/c-code/race-condition$ gcc rc-mutex.c -o rc-mutex -lpthread
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-mutex
Final value of shared_counter: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$
```

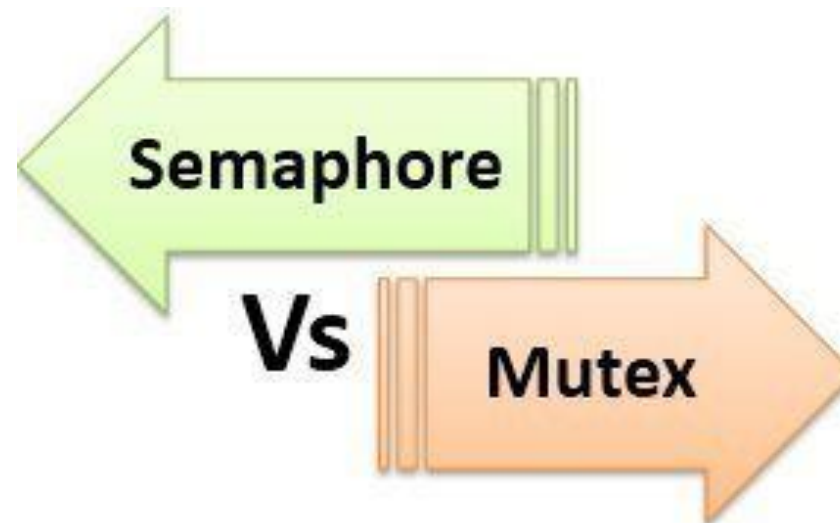
```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define NUM_THREADS 8
5 #define ITERATIONS 100000
6
7 int shared_counter = 0; // Shared variable
8 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; // Mutex for
9 // synchronization
10
11 // Thread function: increments shared_counter ITERATIONS times
12 void* increment_counter(void* arg) {
13     for (int i = 0; i < ITERATIONS; i++) {
14         pthread_mutex_lock(&mutex); // Lock the mutex
15         shared_counter++;
16         pthread_mutex_unlock(&mutex); // Unlock the mutex
17     }
18     return NULL;
19 }
20
21 int main() {
22     pthread_t threads[NUM_THREADS];
23
24     // Create NUM_THREADS threads
25     for (int i = 0; i < NUM_THREADS; i++) {
26         pthread_create(&threads[i], NULL, increment_counter, NULL);
27     }
28
29     // Wait for all threads to finish
30     for (int i = 0; i < NUM_THREADS; i++) {
31         pthread_join(threads[i], NULL);
32     }
33
34     printf("Final value of shared_counter: %d (expected: %d)\n",
35           shared_counter, NUM_THREADS * ITERATIONS);
36
37     return 0;
38 }
39
```

```
1 // pthread mutex example
2 int data=123; // the global shared variable
3 // Thread 1                                // Thread 2
4
5 pthread_mutex_lock(&mut);
6 int a; // local variable
7 a = data;
8 a++;
9 data = a;
10 pthread_mutex_unlock(&mut);
11
12
13
14
```

CRITICAL SECTION

```
pthread_mutex_lock(&mut);
/* blocked */
/* blocked */
/* blocked */
/* blocked */ int b; // local variable
b = data;
b--;
data = b;
pthread_mutex_unlock(&mut);
```

4) MUTEX VS. SEMAPHORE



4) MUTEX VS. SEMAPHORE

BASIS FOR COMPARISON	SEMAPHORE	MUTEX
Basic	Semaphore is a signalling mechanism.	Mutex is a locking mechanism.
Existence 	Semaphore is an integer variable.	Mutex is an object.
Function	Semaphore allow multiple program threads to access a finite instance of resources.	Mutex allow multiple program thread to access a single resource but not simultaneously.
Ownership	Semaphore value can be changed by any process acquiring or releasing the resource.	Mutex object lock is released only by the process that has acquired the lock on it.

BASIS FOR COMPARISON	SEMAPHORE	MUTEX
Categorize	Semaphore can be categorized into counting semaphore and binary semaphore.	Mutex is not categorized further.
Operation	Semaphore value is modified using wait() and signal() operation.	Mutex object is locked or unlocked by the process requesting or releasing the resource.
Resources Occupied	If all resources are being used, the process requesting for resource performs wait() operation and block itself till semaphore count become greater than one.	If a mutex object is already locked, the process requesting for resources waits and queued by the system till lock is released.

4) MUTEX VS. SEMAPHORE

sem_overview(7) — Linux manual page

NAME [top](#)

`sem_overview` - overview of POSIX semaphores

DESCRIPTION [top](#)

POSIX semaphores allow processes and threads to synchronize their actions.

A semaphore is an integer whose value is never allowed to fall below zero. Two operations can be performed on semaphores: increment the semaphore value by one (`sem_post(3)`); and decrement the semaphore value by one (`sem_wait(3)`). If the value of a semaphore is currently zero, then a `sem_wait(3)` operation will block until the value becomes greater than zero.

POSIX semaphores come in two forms: named semaphores and unnamed semaphores.

```

1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4
5 #define NUM_THREADS 8
6 #define ITERATIONS 100000
7
8 int common_var = 0; // Shared variable
9 sem_t sem;         // Semaphore for synchronization
10
11 // Function that each thread will execute
12 void* increment(void* arg) {
13     for (int i = 0; i < ITERATIONS; i++) {
14         sem_wait(&sem);    // Lock: wait if another thread is in critical section
15         common_var++;       // Critical section
16         sem_post(&sem);    // Unlock: signal that critical section is free
17     }
18     return NULL;
19 }
20
21 int main() {
22     pthread_t threads[NUM_THREADS];
23
24     // Initialize semaphore with value 1 (binary semaphore)
25     sem_init(&sem, 0, 1);
26
27     // Create 8 threads
28     for (int i = 0; i < NUM_THREADS; i++) {
29         pthread_create(&threads[i], NULL, increment, NULL);
30     }
31
32     // Wait for all threads to finish
33     for (int i = 0; i < NUM_THREADS; i++) {
34         pthread_join(threads[i], NULL);
35     }
36
37     // Destroy semaphore
38     sem_destroy(&sem);
39
40     printf("Final value of common_var: %d (expected: %d)\n",
41           common_var, NUM_THREADS * ITERATIONS);
42
43     return 0;
44 }
45

```

- Solution with semaphores



```

jerome@jerome-virtualbox:~/c-code/race-condition$ gcc -o rc-sem rc-sem.c -lpthread
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-sem
Final value of common_var: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-sem
Final value of common_var: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-sem
Final value of common_var: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-sem
Final value of common_var: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$ ./rc-sem
Final value of common_var: 800000 (expected: 800000)
jerome@jerome-virtualbox:~/c-code/race-condition$

```


4) MUTEX VS. SEMAPHORE

- Producer/Consumer problems
- pthread_cond:

```
#include <pthread.h>

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);

int pthread_cond_signal(pthread_cond_t *cond);

int pthread_cond_broadcast(pthread_cond_t *cond);

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);

int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);

int pthread_cond_destroy(pthread_cond_t *cond);
```

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4
5 #define NUM_THREADS 8
6 #define ITERATIONS 10000000
7
8 int counter = 0;
9 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
10 pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
11
12 void* increment(void* arg) {
13     for (int i = 0; i < ITERATIONS; i++) {
14         pthread_mutex_lock(&mutex);
15         counter++;
16         pthread_cond_signal(&cond); // Not needed here, just for demonstration
17         pthread_mutex_unlock(&mutex);
18     }
19     return NULL;
20 }
21
```

```
21
22 int main() {
23     pthread_t threads[NUM_THREADS];
24
25     for (int i = 0; i < NUM_THREADS; i++) {
26         if (pthread_create(&threads[i], NULL, increment, NULL) != 0) {
27             perror("Failed to create thread");
28             exit(EXIT_FAILURE);
29         }
30     }
31
32     for (int i = 0; i < NUM_THREADS; i++) {
33         if (pthread_join(threads[i], NULL) != 0) {
34             perror("Failed to join thread");
35             exit(EXIT_FAILURE);
36         }
37     }
38
39     pthread_mutex_destroy(&mutex);
40     pthread_cond_destroy(&cond);
41
42     printf("Expected: %d, Actual: %d\n", NUM_THREADS * ITERATIONS, counter);
43     return 0;
44 }
45
```

4) MUTEX VS. SEMAPHORE

- Classic **producer-consumer** example in C using `pthread_cond_signal` to synchronize access to a shared buffer.
- This example demonstrates how producers and consumers can work together using a mutex and condition variables to avoid race conditions and busy waiting.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 5

// Shared buffer and its current size
int buffer[BUFFER_SIZE];
int buffer_index = 0;

// Mutex to protect the buffer
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
// Condition variables for producers and consumers
pthread_cond_t can_produce = PTHREAD_COND_INITIALIZER;
pthread_cond_t can_consume = PTHREAD_COND_INITIALIZER;
```

```
// Producer function
void* producer(void* arg) {
    int item;
    for (int i = 0; i < 20; i++) {
        item = rand() % 100; // Produce a random item

        pthread_mutex_lock(&mutex);

        // Wait until there is space in the buffer
        while (buffer_index == BUFFER_SIZE) {
            printf("Producer waiting: buffer full\n");
            pthread_cond_wait(&can_produce, &mutex);
        }

        // Add the item to the buffer
        buffer[buffer_index++] = item;
        printf("Produced: %d (Buffer size: %d)\n", item, buffer_index);

        // Signal that the buffer is no longer empty
        pthread_cond_signal(&can_consume);

        pthread_mutex_unlock(&mutex);

        // Simulate some work
        usleep(rand() % 100000);
    }
    return NULL;
}
```

```
// Consumer function
void* consumer(void* arg) {
    int item;
    for (int i = 0; i < 20; i++) {
        pthread_mutex_lock(&mutex);

        // Wait until there is an item in the buffer
        while (buffer_index == 0) {
            printf("Consumer waiting: buffer empty\n");
            pthread_cond_wait(&can_consume, &mutex);
        }

        // Remove an item from the buffer
        item = buffer[--buffer_index];
        printf("Consumed: %d (Buffer size: %d)\n", item, buffer_index);

        // Signal that the buffer is no longer full
        pthread_cond_signal(&can_produce);

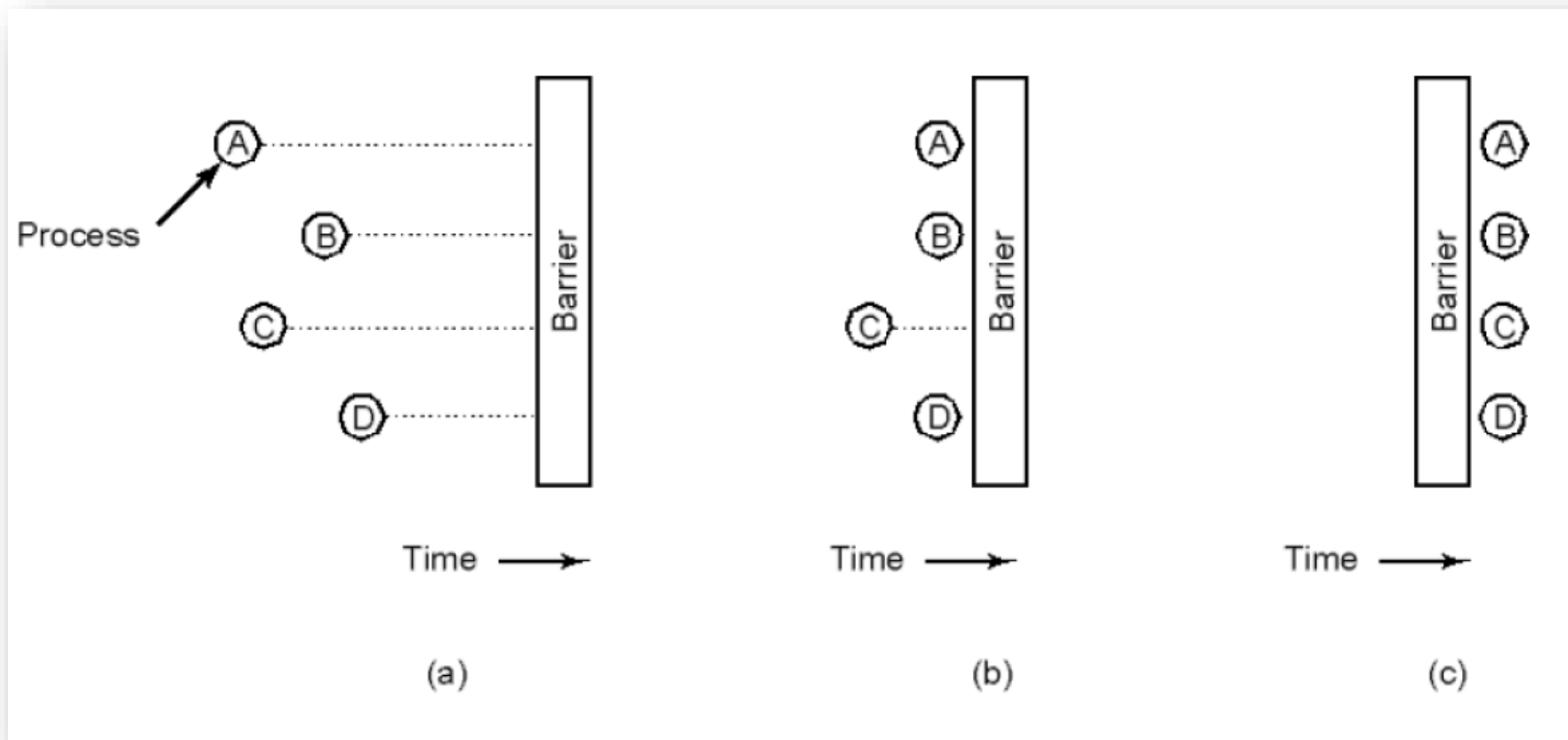
        pthread_mutex_unlock(&mutex);

        // Simulate some work
        usleep(rand() % 100000);
    }
    return NULL;
}
```

```
int main() {  
    pthread_t producer_thread, consumer_thread;  
  
    // Create producer and consumer threads  
    pthread_create(&producer_thread, NULL, producer, NULL);  
    pthread_create(&consumer_thread, NULL, consumer, NULL);  
  
    // Wait for threads to finish  
    pthread_join(producer_thread, NULL);  
    pthread_join(consumer_thread, NULL);  
  
    // Cleanup  
    pthread_mutex_destroy(&mutex);  
    pthread_cond_destroy(&can_produce);  
    pthread_cond_destroy(&can_consume);  
  
    return 0;  
}
```


5) SYNCHRONIZATION OF THREADS

- `pthread_barrier`:



5) SYNCHRONIZATION OF THREADS

- pthread_barrier to compute PI:

$$\pi = 4 \left(\sum_{k=0}^{\infty} \frac{1}{4k+1} - \sum_{k=0}^{\infty} \frac{1}{4k+3} \right)$$

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <stdlib.h>
4
5 #define NUM_THREADS 8
6 #define ITERATIONS 1000000
7
8 double positive_sum = 0.0;
9 double negative_sum = 0.0;
10 double result = 0.0;
11 pthread_barrier_t barrier;
12
13 typedef struct {
14     int id;
15     int num_threads;
16 } ThreadData;
17
```

```
17
18 // Function for threads computing the positive part
19 void* compute_positive(void* arg) {
20     ThreadData* data = (ThreadData*)arg;
21     int start = data->id * (ITERATIONS / (NUM_THREADS/2));
22     int end = (data->id + 1) * (ITERATIONS / (NUM_THREADS/2));
23     double local_sum = 0.0;
24
25     for (int k = start; k < end; k++) {
26         local_sum += 1.0 / (4*k + 1);
27     }
28     positive_sum += local_sum;
29
30     pthread_barrier_wait(&barrier);
31     return NULL;
32 }
33
```

```
33
34 // Function for threads computing the negative part
35 void* compute_negative(void* arg) {
36     ThreadData* data = (ThreadData*)arg;
37     int start = data->id * (ITERATIONS / (NUM_THREADS/2));
38     int end = (data->id + 1) * (ITERATIONS / (NUM_THREADS/2));
39     double local_sum = 0.0;
40
41     for (int k = start; k < end; k++) {
42         local_sum += 1.0 / (4*k + 3);
43     }
44     negative_sum += local_sum;
45
46     pthread_barrier_wait(&barrier);
47     return NULL;
48 }
49
```

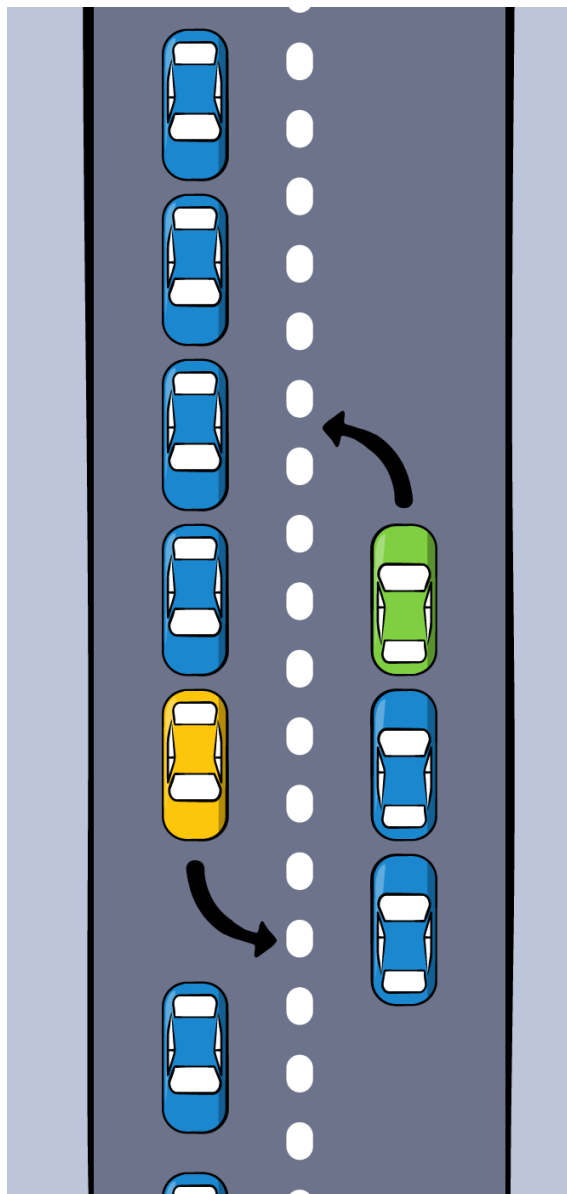
```
49
50 int main() {
51     pthread_t threads[NUM_THREADS];
52     ThreadData thread_data[NUM_THREADS];
53
54     pthread_barrier_init(&barrier, NULL, NUM_THREADS);
55
56     // Create threads for positive and negative parts
57     for (int i = 0; i < NUM_THREADS/2; i++) {
58         thread_data[i].id = i;
59         thread_data[i].num_threads = NUM_THREADS;
60         pthread_create(&threads[i], NULL, compute_positive, &thread_data[i]);
61     }
62     for (int i = NUM_THREADS/2; i < NUM_THREADS; i++) {
63         thread_data[i].id = i - NUM_THREADS/2;
64         thread_data[i].num_threads = NUM_THREADS;
65         pthread_create(&threads[i], NULL, compute_negative, &thread_data[i]);
66     }
67
68     // Wait for all threads to finish
69     for (int i = 0; i < NUM_THREADS; i++) {
70         pthread_join(threads[i], NULL);
71     }
72
73     // Combine results after barrier
74     result = 4.0 * (positive_sum - negative_sum);
75
76     printf("Computed value of PI: %f\n", result);
77
78     pthread_barrier_destroy(&barrier);
79     return 0;
80 }
81
```

```
jerome@jerome-virtualbox:~/c-code/race-condition$ gcc -o threads-barrier threads-barrier.c -lpthread -lm
jerome@jerome-virtualbox:~/c-code/race-condition$ ./threads-barrier
Computed value of PI: 3.141592
jerome@jerome-virtualbox:~/c-code/race-condition$ ./threads-barrier
Computed value of PI: 3.141592
jerome@jerome-virtualbox:~/c-code/race-condition$ ./threads-barrier
Computed value of PI: 3.141592
jerome@jerome-virtualbox:~/c-code/race-condition$ ./threads-barrier
Computed value of PI: 3.141592
jerome@jerome-virtualbox:~/c-code/race-condition$ ./threads-barrier
Computed value of PI: 3.141592
jerome@jerome-virtualbox:~/c-code/race-condition$
```

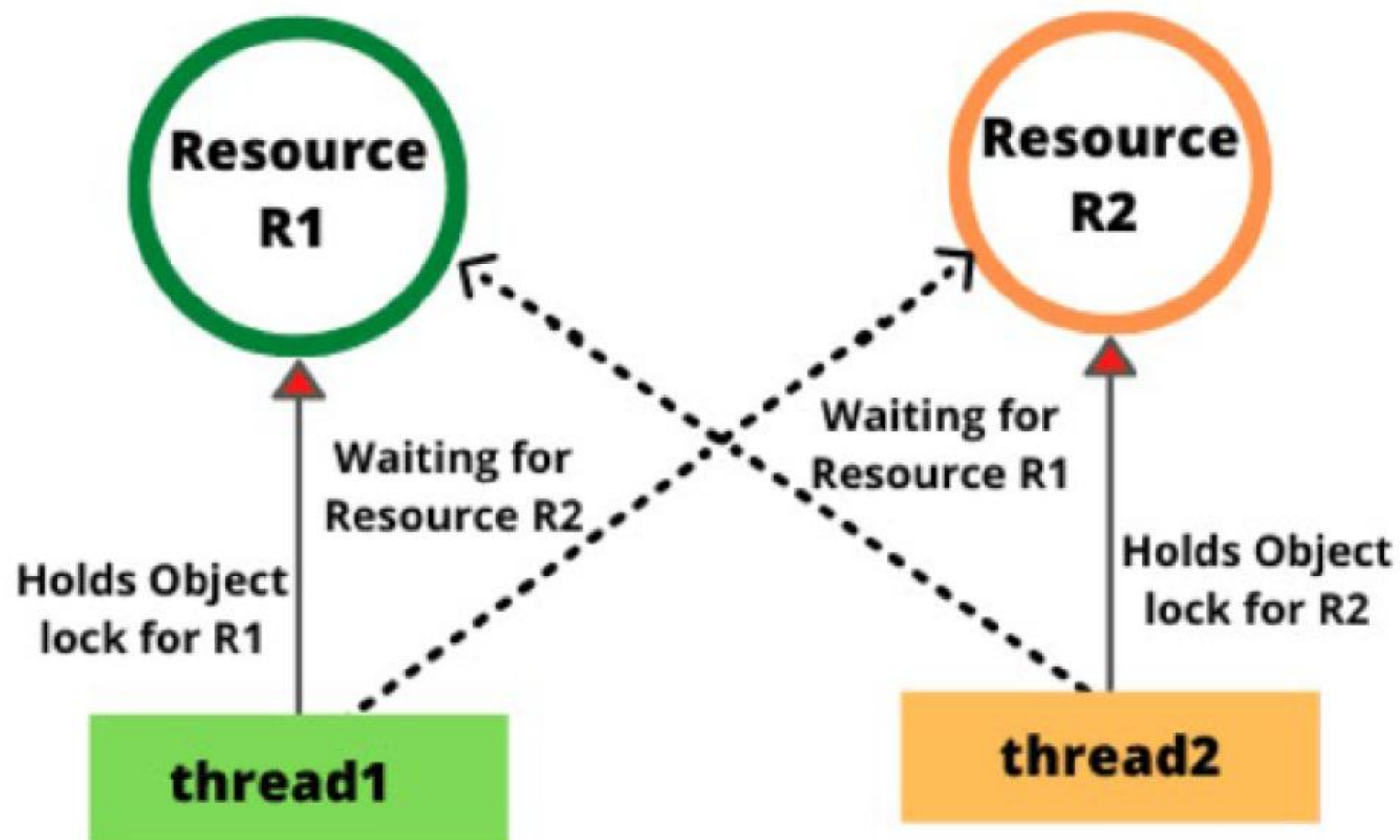
6) DEADLOCK

When two or more processes stop making progress *indefinitely* because they are all waiting for each other to do something.

6) DEADLOCK



6) DEADLOCK



Here, both threads are waiting for each other to unlock resources R1 and R2, but thread2 cannot release lock for resource R2 until it gets hold of resource R1.

6) DEADLOCK

- Say you have two processes P1 and P2
- Both need to acquire two locks L1 and L2 to access a resource.
- **Problem: Deadlock**
 - P1 acquires L1
 - P2 acquires L2
 - P1 tries to acquire L2 and blocks
 - P2 tries to acquire L1 and blocks
 - We have a **deadlock**!
- **Solution: Lock Ordering**
 - Sort the locks in a fixed order (say L1 followed by L2)
 - Always acquire locks in the sorted order.
- Lock ordering example:
 - P1 acquires L1
 - P2 tries to acquire L1 and blocks
 - P1 acquires L2
 - P1 executes critical section
 - P1 releases L2
 - P1 releases L1
 - P2 wakes up
 - P2 acquires L1
 - P2 acquires L2
 - P2 executes critical section
 - P2 releases L2
 - P2 releases L1
 - No deadlock!

6) DEADLOCK

- General solution for N-number of locks
 - Acquire locks in the same order in all threads
 - Release the locks in the reverse order

6) DEADLOCK

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

(a)

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

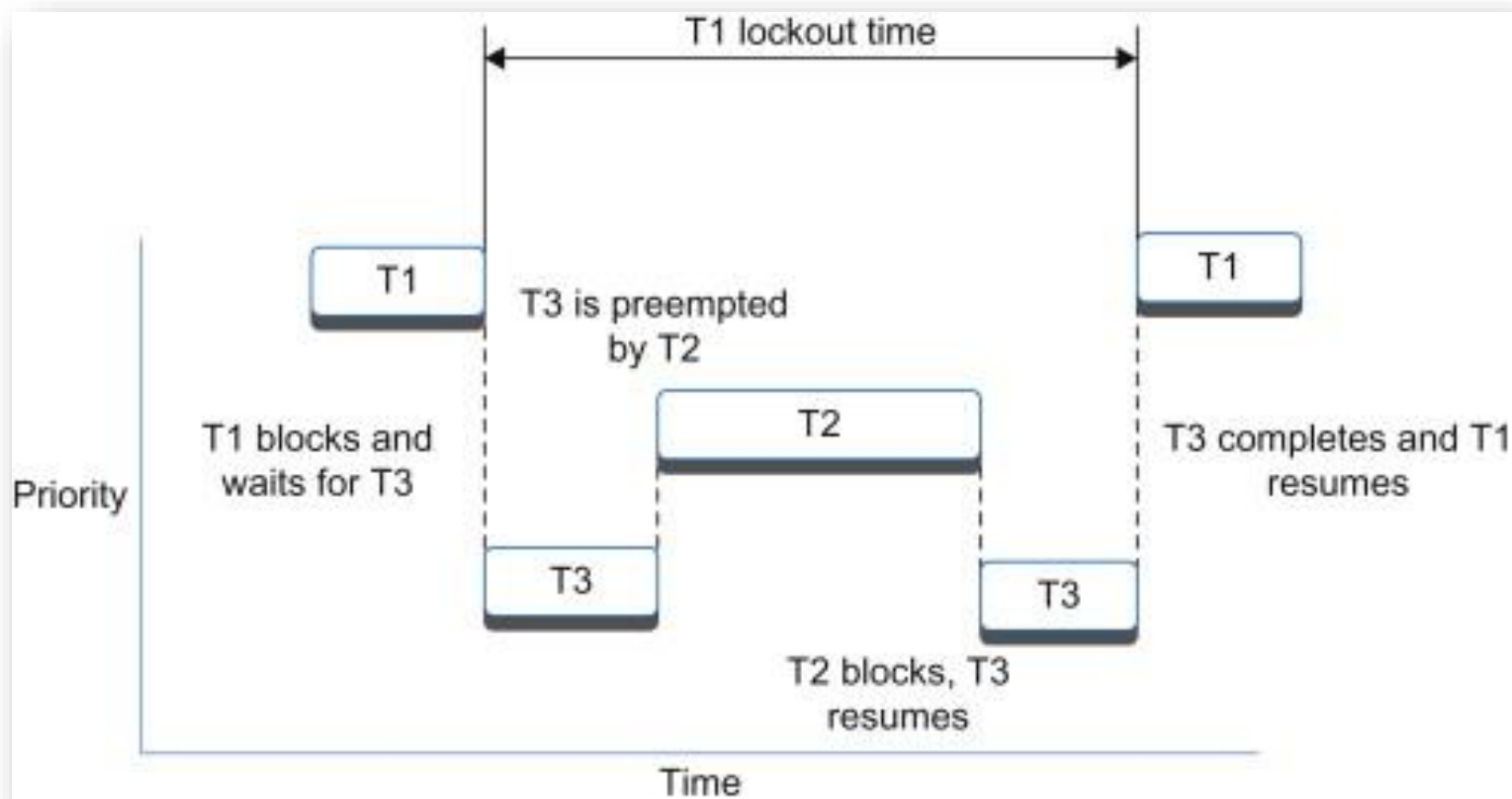
```
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

(b)

Figure 6-2. (a) Deadlock-free code. (b) Code with a potential deadlock.

7) PRIORITY INVERSION

- Priority inversion when the priorities solves a deadlock:



ANY QUESTIONS?